

实验报告

课程：数字逻辑与计算机组成实验

实验八：CPU 数据通路

姓名： 郑袭明

学号： 251502027

日期：2026 年 5 月 22 日

目录

1	设计思路	2
1.1	整体架构	2
1.2	算术逻辑单元 (ALU) 设计	2
1.3	数据存储器 (DMEM) 设计	3
2	实验总结与反思	3

1 设计思路

1.1 整体架构

本实验实现了 CPU 数据通路中的两个核心部件：算术逻辑单元（ALU）和数据存储器（DMEM）。ALU 负责执行各种算术与逻辑运算，并输出计算结果以及用于分支判断的信号；DMEM 则用于数据存储，能够支持字节（Byte）、半字（Halfword）和字（Word）级别的读写访问。

在板级验证中，ALU 和 DMEM 通过各自的顶层模块 `top` 分别连接至板载的拨码开关、LED 和七段数码管。拨码开关用于模拟输入数据及控制信号，LED 和数码管用于实时观察模块输出的正确性。

1.2 算术逻辑单元 (ALU) 设计

ALU 模块的核心任务是根据 4 位控制信号 `ALUctr` 对两个 32 位操作数 `dataa` 和 `datab` 进行运算，接口信号如图 1 所示。



图 1: ALU 模块接口信号图

其具体设计特点如下：

- 多功能运算支持：**利用 Verilog 的条件分支选择器实现了加法、减法、逻辑左移、逻辑右移、算术右移、按位与、按位或、按位异或以及直接输出等多种运算。
- 有符号与无符号比较：**通过 `ALUctr[3]` 信号区分有符号与无符号比较操作，当执行比较时，将比较结果输出到 `less`。
- 分支决策辅助：**当 `ALUctr[2:0]` 为等值比较指令对应的编码时，`zero` 信号在两输入相等（即 `dataa == datab`）时输出高电平，便于分支指令（如 `beq / bne`）进行条件跳转判断；而在其他运算下，`zero` 信号表示运算结果 `alurestult` 是否为零。
- 移位位数截断：**移位指令的移动位数仅取 `datab` 的低 5 位（即 `datab[4:0]`），契合 RISC-V 32 位架构的规范。

1.3 数据存储单元 (DMEM) 设计

DMEM 模块利用 Vivado 的 Block Memory Generator IP 核（配置为 Simple Dual Port RAM）来存储数据。为了实现 RISC-V 中对字节、半字和字等不同宽度的加载与存储指令，设计了非对齐与宽度格式化的对齐逻辑，其接口信号如图 2 所示。



图 2: DMEM 模块接口信号图

DMEM 访存处理流程设计如下：

1. **带字节写使能的存储逻辑 (Write Enable Masking):** 为 IP 级 RAM 启用了字节写使能功能。根据输入的控制信号 `memop` 和地址的低 2 位 `byte_offset = addr[1:0]`，生成 4 位的写使能掩码 `wea`，同时将待写入的数据 `datain` 移位对齐到对应的字节或半字位置，生成 `dina` 送入 IP 核。这极大地简化了 `sb` (Store Byte) 和 `sh` (Store Halfword) 指令的处理，省去了先读出整字、拼接再写入的低效步骤。
2. **对齐与数据扩展的加载逻辑 (Read Formatting):** 从双端口 BRAM 读出的 32 位整字数据 `doutb` 需要根据 `byte_offset` 提取出目标字节或半字。对于有符号加载指令 (LB、LH)，根据提取部分的高位进行符号位扩展 (Sign Extension)；对于无符号加载指令 (LBU、LHU)，则进行零扩展 (Zero Extension)；对于字加载指令 (LW)，则直接输出整个 32 位字。

系统通过 `memop[2:0]` 区分操作类型，其编码定义如下：

- 3'b000: 有符号字节读写 (LB / SB)
- 3'b001: 有符号半字读写 (LH / SH)
- 3'b010: 字读写 (LW / SW)
- 3'b100: 无符号字节读 (LBU)
- 3'b101: 无符号半字读 (LHU)

2 实验总结与反思

本次实验相较于最近几次实验来说非常顺利，没有遇到什么问题。